

IBM University Engagement Project Submission

*Predictive Maintenance of Industrial Machinery — Machine Learning
Classification*

Project Tile – Predictive Maintenance of Industrial Machinery

Domain of Project – Mechanical Engineering / Predictive Maintenance

Student Name	Recent Passport Photo	Email ID	Mobile number [WhatsApp]
Vidhya Majee		vidhyamajee@gmail.com	8052134234

Recommended font style and size:

Font Style: Arial

Font Size (Heading): 28

Font Size (Content): 20

Problem Statement -

Problem Statement : Predictive Maintenance of Industrial Machinery

The Challenge

Unexpected failures in industrial machinery cause costly downtime, safety risks, and unplanned maintenance expenses. The goal is to **anticipate failures before they occur** by analyzing sensor data from a fleet of industrial machines to identify patterns that precede a failure, enabling **proactive maintenance and reduced operational costs**.

The Objective

Build a **Machine Learning classification model** that predicts the **type of failure** based on real-time operational sensor data. The solution should:

- **Multi-Class Failure Prediction** - Classify sensor readings into 6 categories: No Failure, Heat Dissipation Failure, Power Failure, Overstrain Failure, Tool Wear Failure, and Random Failures.
- **Sensor Data Analysis** - Use air temperature, process temperature, rotational speed, torque, and tool wear readings from real-time operational data.
- **Class Imbalance Handling** - Address the severe real-world imbalance (96.5% "No Failure" samples) using stratified splitting and class-weighted training.
- **Model Comparison** - Establish a baseline (Decision Tree) and compare against stronger ensemble models (Random Forest, XGBoost) to identify the most reliable classifier.
- **Explainability** - Use SHAP analysis to reveal which sensor readings drive each failure prediction, keeping the model transparent to maintenance engineers.
- **Dataset** - Machine Predictive Maintenance Classification dataset (Kaggle), 10,000 sensor readings across 6 failure classes.

Proposed Solution -

Proposed Solution: ML-Based Predictive Maintenance Classifier

The proposed solution is a **multi-class Machine Learning classification pipeline** built using **Python, scikit-learn, and XGBoost** to predict industrial machine failure types from real-time sensor data. The system addresses the challenge of unplanned downtime through an interpretable, data-driven pipeline.

At its core, the solution follows a structured **three-stage pipeline**: preprocessing, model training, and explainability. Sensor readings (air temperature, process temperature, rotational speed, torque, and tool wear) are cleaned, encoded, and split using stratified sampling to preserve the real-world class distribution.

The solution trains and compares three models:

Decision Tree (Baseline) establishes a reference point, revealing that high accuracy alone can mask poor performance on rare failure classes.

Random Forest with balanced class weights significantly improves detection of minority failure types.

XGBoost emerges as the best-performing model, achieving the strongest macro F1-score across all six failure categories.

The solution includes a **SHAP explainability layer** that visualizes which sensor readings (e.g., Torque, Tool Wear) drive each failure prediction, along with a confusion matrix that highlights exactly where the model succeeds and where it struggles.

IBM Bob was used as an AI development tool to assist with code review and documentation throughout the project.

Overall, this solution delivers an **interpretable, data-driven predictive maintenance assistant** that helps maintenance engineers anticipate failures before they occur, reducing downtime and operational costs.

Technology Used

- **Python** - Core programming language for the entire ML pipeline
- **scikit-learn** - Decision Tree baseline, Random Forest classifier, train-test splitting, and evaluation metrics
- **XGBoost** - Best-performing gradient boosting classifier for multi-class failure prediction
- **SHAP** - Model explainability library, showing per-class feature importance for maintenance engineers
- **matplotlib / seaborn** - Visualizations including confusion matrix and SHAP summary plots
- **IBM Bob** - AI development tool used for code review and documentation assistance
- Note: IBM Cloud Lite and IBM Bob was used as the accessible IBM tooling for this project.

Files Reviewed Using IBM Bob -

File Edit Selection View Go Run Terminal Help

predictive-maintenance-ml

predictive_maintenance.ipynb

predictive_maintenance.ipynb > import pandas as pd

+ Code + Markdown | Run All

Select Kernel

Python

df = pd.read_csv('predictive_maintenance.csv')

Python

print("Shape:", df.shape)

df.head()

Python

Shape: (10000, 10)

UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	1	M14860	M	298.1	308.6	1551	42.8	0	No Failure
1	2	L47181	L	298.2	308.7	1408	46.3	3	No Failure
2	3	L47182	L	298.1	308.5	1498	49.4	5	No Failure
3	4	L47183	L	298.2	308.6	1433	39.5	7	No Failure

EXPLORED

PREDICTIVE-MAINTENANCE-ML

.ipynb_checkpoints

.virtual_documents

anaconda_projects

.gitignore

confusion_matrix.png

predictive_maintenance.csv

predictive_maintenance.ipynb

README.md

shap_summary.png

OUTLINE

TIMELINE

IBM BOB

Review my predictive_mainte... 21.3k / 270.0k 0.135

Review my predictive_maintenance.ipynb notebook and summarize the ML approach used, including the models trained and key findings from the SHAP analysis.

Explored 4 files : 6s

Now I have a thorough understanding of the entire notebook. Let me provide a comprehensive summary.

Here's a full review of the predictive_maintenance.ipynb notebook:

Dataset

- Source file: predictive_maintenance.csv
- Size: 10,000 rows x 10 columns
- Features: Air temperature [K], Process temperature [K], Rotational speed [rpm], Torque [Nm], Tool wear [min], and product Type (L/M/H)
- Target: Failure Type — a 6-class multiclass label
- Class distribution (highly imbalanced):

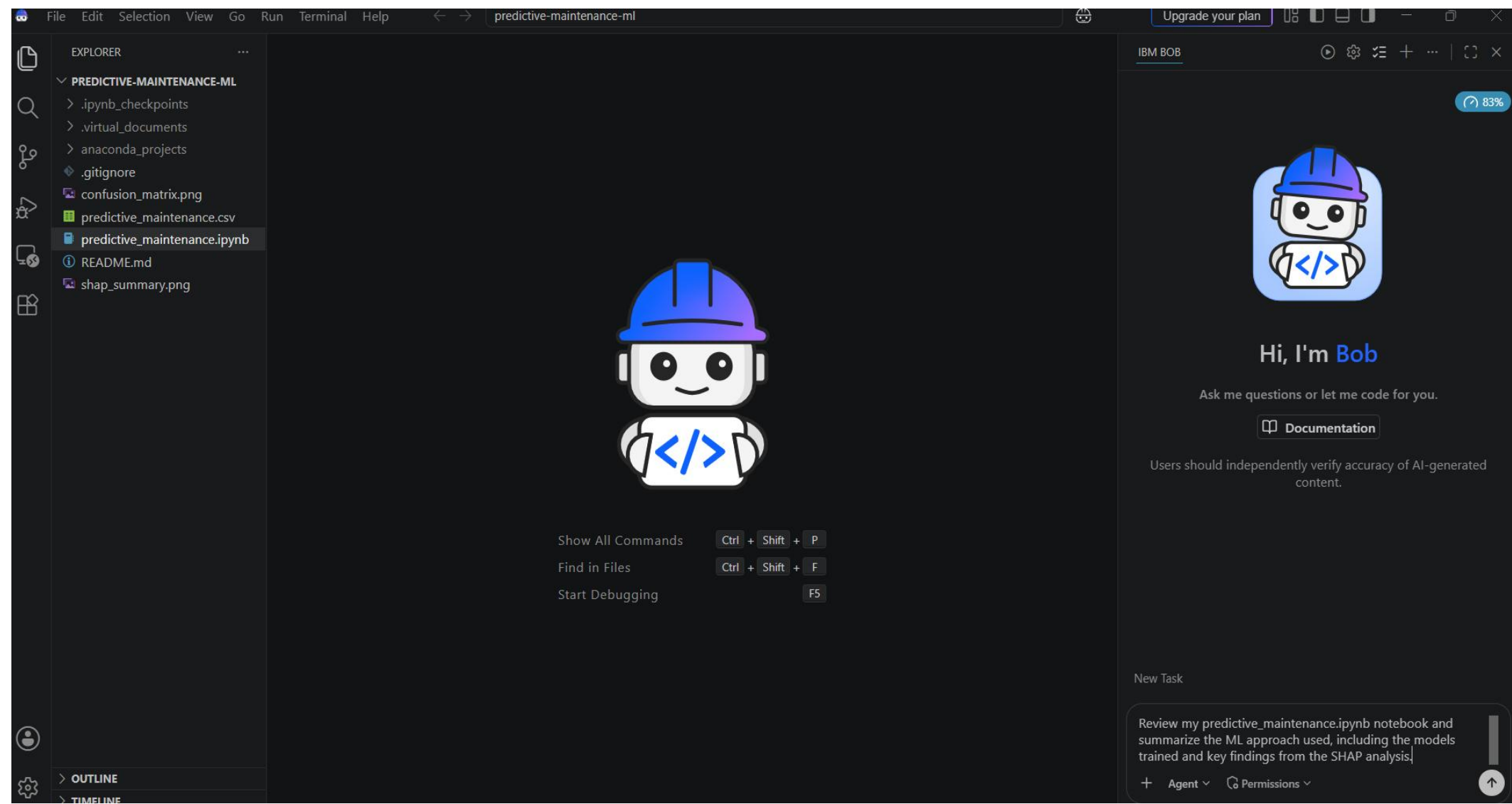
Class	Count
No Failure	9,652
Heat Dissipation Failure	112

Understood, working on it...

Follow up, ⌘ to steer, Alt ⌘ to queue, esc to cancel

+ Agent Permissions

IBM Bob Screenshots:



FileEditSelectionViewGoRunTerminalHelp

predictive-maintenance-ml

Upgrade your plan

EXPLORER

PREDICTIVE-M...

.ipynb_checkpoints

.virtual_documents

anaconda_projects

.gitignore

confusion_matrix.png

predictive_maintenance.csv

predictive_maintenance.ipynb

README.md

shap_summary.png

OUTLINE

TIMELINE

predictive_maintenance.ipynb

shap_summary.png

predictive_maintenance.ipynb

import pandas as pd

cm = confusion_matrix(y_test_, y_pred_)

plt.figure(figsize=(10, 8))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',

xticklabels=le.classes_, yticklabels=le.classes_)

plt.xlabel('Predicted')

plt.ylabel('Actual')

plt.title('XGBoost Confusion Matrix')

plt.tight_layout()

plt.savefig('confusion_matrix.png', dpi=150)

plt.show()

[15]

Python

XGBoost Confusion Matrix

Heat Dissipation Failure -	19	3	0	0	0	0
No Failure -	1	1922	1	1	0	5
Overstrain Failure -	0	2	14	0	0	0
Power Failure -	0	4	0	15	0	0
Random Failures -	0	4	0	0	0	0
Tool Wear Failure -	0	8	1	0	0	0

Actual

IBM BOB

Review my predictive mainten... 30.0k / 270.0k 0.195

which sensor readings drive each failure type prediction most strongly. Based on the domain and model behavior, Torque [Nm], Tool wear [min], and Rotational speed [rpm] are the features most likely to dominate — particularly for Overstrain, Heat Dissipation, and Power failures. The Type_* dummies likely contribute less globally but may matter for specific failure modes.

Key Observations

1. Class imbalance is the central challenge.

With 96.5% of samples being "No Failure", even a model that predicts nothing else gets 96.5% accuracy. Macro F1 is the meaningful metric here.

2. XGBoost is the clear winner across macro F1,

benefiting from its handling of imbalanced distributions and its ensemble depth. Random Forest with class_weight="balanced" was a solid improvement over the shallow Decision Tree baseline.

3. Random Failures and Tool Wear Failure remain undetected

by all models — their sample counts (18 and 45) are likely too small for reliable learning without oversampling (e.g., SMOTE) or threshold tuning.

4. The SHAP analysis provides class-level explainability —

a valuable step beyond standard feature importances, enabling maintenance engineers to understand which sensor readings indicate which failure mode.

New task

Follow up or start new task Ctrl+N

+

Agent

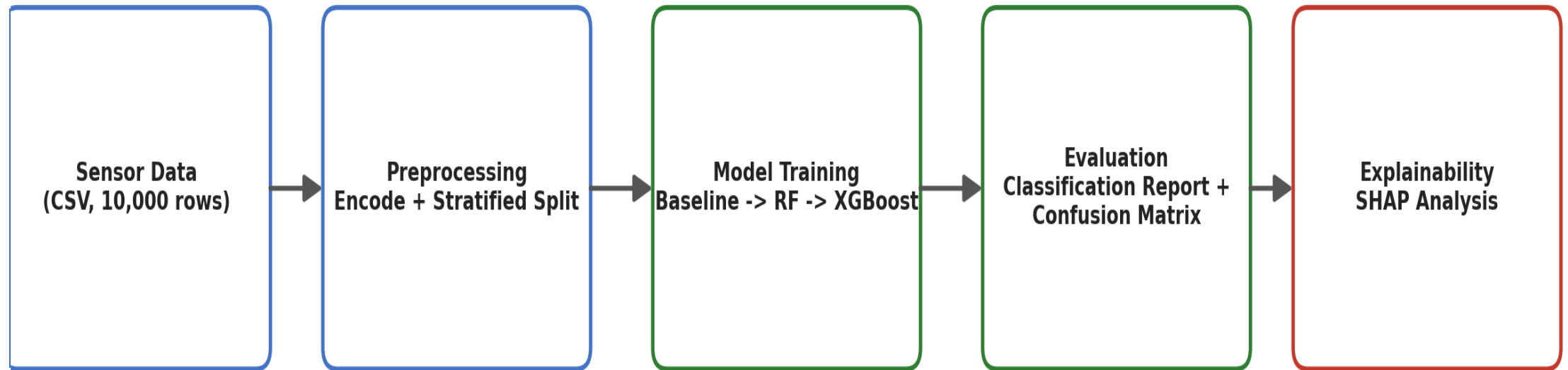
Permissions

↑

Cell 1 of 14

Bob Settings

ML Pipeline Architecture:



Role of AI Tooling in the Solution

Role of AI Tooling in Predictive Maintenance

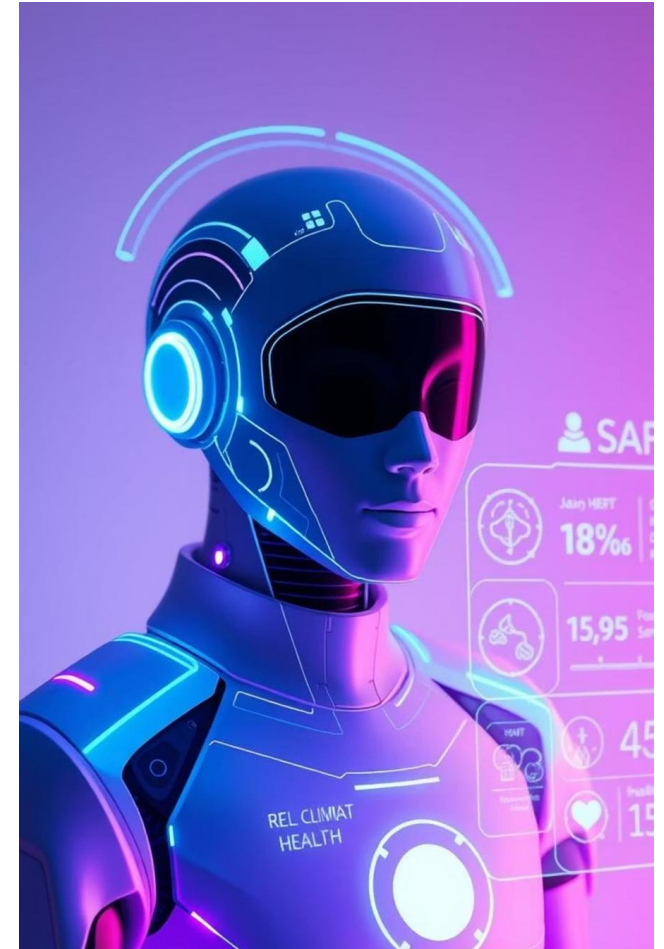
This project uses **supervised Machine Learning** - not Agentic AI - to solve a multi-class failure classification problem. **IBM Bob** was used as an AI-assisted development tool to support code review, documentation, and understanding of the notebook's structure.

The core intelligence of this solution comes from the trained **XGBoost classifier** and its **SHAP-based explainability layer**, which together allow the system to not only predict failures but also explain which sensor readings drove each prediction.

This gives maintenance engineers **actionable, interpretable insights** rather than a black-box alert, helping bridge the gap between raw sensor data and confident maintenance decisions.

IBM Bob accelerated development by reviewing code quality, summarizing model results, and helping structure clear project documentation.

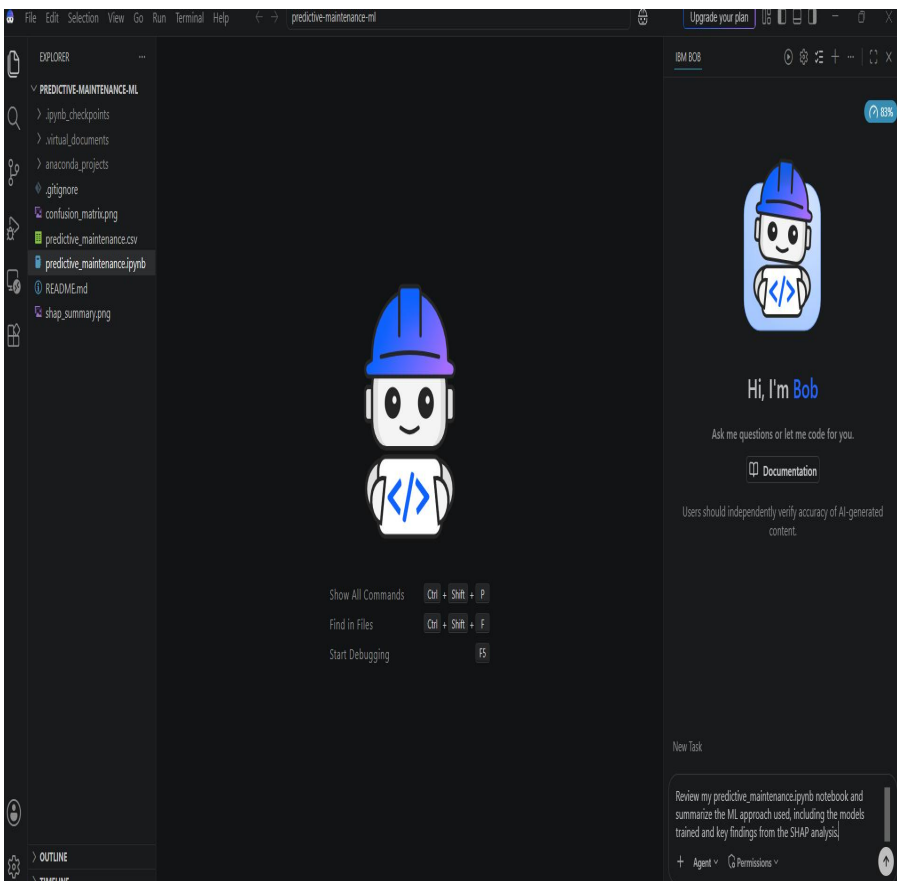
Overall, this approach makes the system transparent, evidence-based, and genuinely useful for real-world industrial maintenance decisions.



Token Usage

30.0k of 270.0k tokens used (0.195 Bobcoins) across the session

Early in Session



IBM BOB

Hi, I'm Bob

Ask me questions or let me code for you.

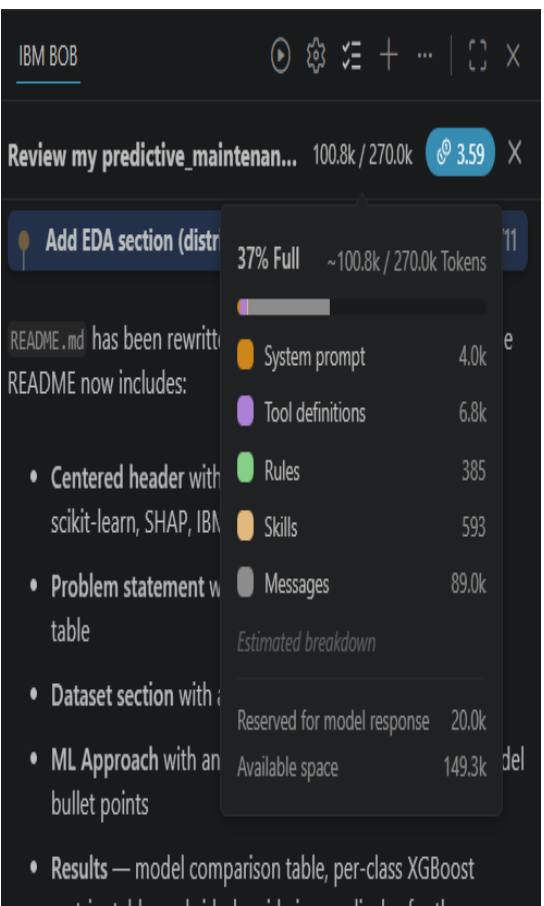
Documentation

Users should independently verify accuracy of AI-generated content.

New Task

Review my predictive_maintenance.ipynb notebook and summarize the ML approach used, including the models trained and key findings from the SHAP analysis.

Later in Session



Review my predictive_maintenance.ipynb notebook 100.8k / 270.0k 3.59

Add EDA section (distributed)

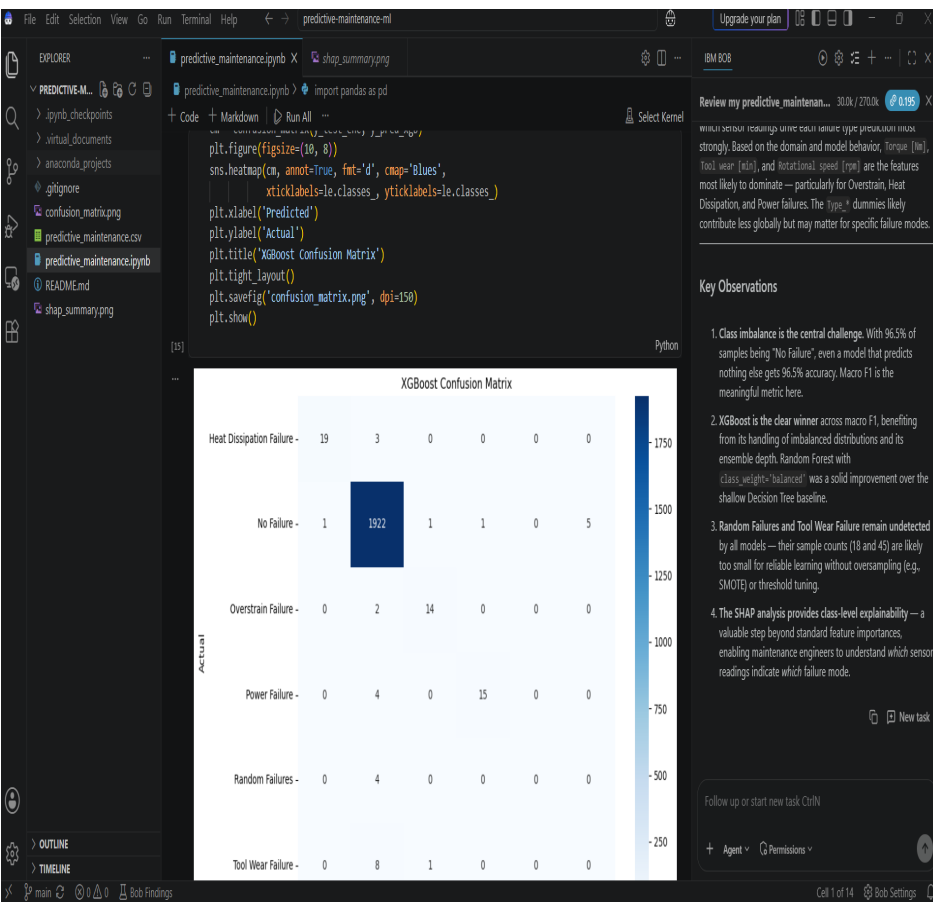
37% Full ~100.8k / 270.0k Tokens

README.md has been rewritten. The README now includes:

- Centered header with scikit-learn, SHAP, IBM
- Problem statement with table
- Dataset section with
- ML Approach with an
- Results — model comparison table, per-class XGBoost

Estimated breakdown

System prompt	4.0k
Tool definitions	6.8k
Rules	385
Skills	593
Messages	89.0k
Reserved for model response	20.0k
Available space	149.3k



Review my predictive_maintenance.ipynb notebook 30.0k / 270.0k 0.195

Which sensor readings drive each failure type prediction most strongly? Based on the domain and model behavior, Torque [Nm], Tool wear [min], and Rotational speed [rpm] are the features most likely to dominate — particularly for Overstrain, Heat Dissipation, and Power failures. The Type_1 dummies likely contribute less globally but may matter for specific failure modes.

Key Observations

- Class imbalance is the central challenge. With 96.5% of samples being "No Failure", even a model that predicts nothing else gets 96.5% accuracy. Macro F1 is the meaningful metric here.
- XGBoost is the clear winner across macro F1, benefiting from its handling of imbalanced distributions and its ensemble depth. Random Forest with `class_weight='balanced'` was a solid improvement over the shallow Decision Tree baseline.
- Random Failures and Tool Wear Failure remain undetected by all models — their sample counts (18 and 45) are likely too small for reliable learning without oversampling (e.g., SMOTE) or threshold tuning.
- The SHAP analysis provides class-level explainability — a valuable step beyond standard feature importances, enabling maintenance engineers to understand which sensor readings indicate which failure mode.

XGBoost Confusion Matrix

	Heat Dissipation Failure	No Failure	Overstrain Failure	Power Failure	Random Failures	Tool Wear Failure
Heat Dissipation Failure	19	3	0	0	0	0
No Failure	1	1922	1	1	0	5
Overstrain Failure	0	2	14	0	0	0
Power Failure	0	4	0	15	0	0
Random Failures	0	4	0	0	0	0
Tool Wear Failure	0	8	1	0	0	0

Project Output Screenshot

FileEditSelectionViewGoRunTerminalHelp

predictive-maintenance-ml

Upgrade your plan

EXPLORER

PREDICTIVE-M...

.ipynb_checkpoints

.virtual_documents

anaconda_projects

.gitignore

confusion_matrix.png

predictive_maintenance.csv

predictive_maintenance.ipynb

README.md

shap_summary.png

OUTLINE

TIMELINE

predictive_maintenance.ipynb

predictive_maintenance.ipynb > import pandas as pd

+ Code+ MarkdownRun All

Select Kernel

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Target	Failure Type
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	No Failure
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	No Failure
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	No Failure
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	No Failure
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	No Failure

df.info()

Python

[5]

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10000 entries, 0 to 9999

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
0	UDI	10000 non-null	int64
1	Product ID	10000 non-null	object
2	Type	10000 non-null	object
3	Air temperature [K]	10000 non-null	float64
4	Process temperature [K]	10000 non-null	float64
5	Rotational speed [rpm]	10000 non-null	int64
6	Torque [Nm]	10000 non-null	float64
7	Tool wear [min]	10000 non-null	int64
8	Target	10000 non-null	int64
9	Failure Type	10000 non-null	object

dtypes: float64(3), int64(4), object(3)

IBM BOB

Review my predictive_mainte... 43.8k / 270.0k 0.674

scoring= t1_macro)

print(f"CV Macro F1: {scores.mean():.3f} ± {scores.std():.3f}")

Medium Priority — Strengthens the Project

5. No Precision-Recall Curves

Tools awaiting approval:

Update Todo List

Edit

Add EDA section (distribution plots, box plots, corr...

Add feature engineering (Power [W], Temp_diff [K]...

Add SMOTE oversampling to fix Tool Wear & Rand...

Add Stratified K-Fold cross-validation

Add Binary Stage-1 model (will it fail?)

Add Precision-Recall curves per class

Add SHAP beeswarm + force plots

Add cost-benefit / business impact analysis

Add IBM Watson Studio markdown cell

Add model saving + real-time inference demo

Approve once

Reject

Approve todo tools for task

1 file changed

Undo allShow all

Follow up, ↵ to steer, Alt ↵ to queue, esc to cancel

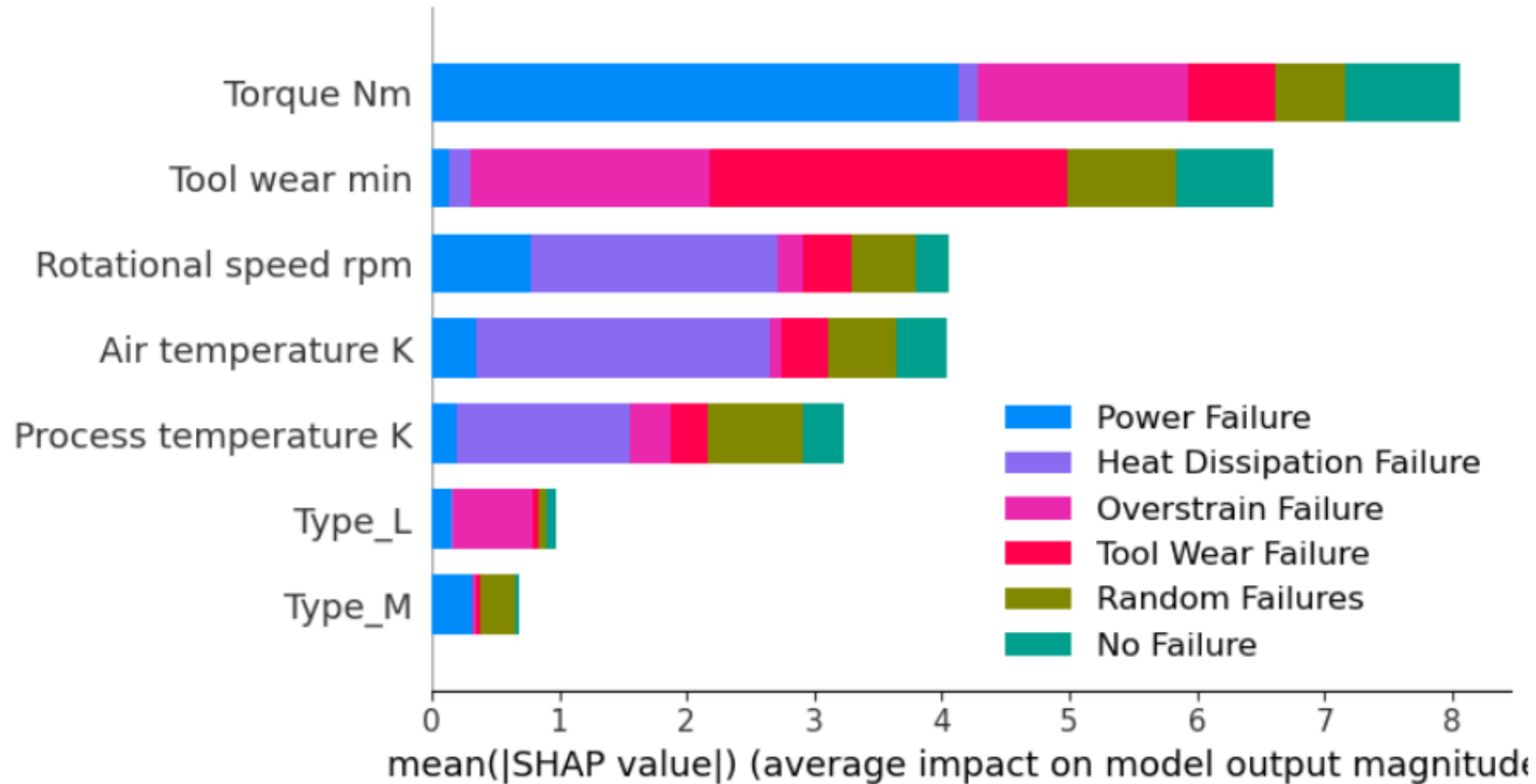
+ Agent

Permissions

Project Output Screenshot



Project Output Screenshot



Novelty and Uniqueness

This project stands out through its rigorous, transparent approach to a real-world **imbalanced classification** problem.

Honest Baseline Comparison - Rather than jumping straight to a complex model, a baseline Decision Tree establishes a genuine reference point, revealing that 97% accuracy can still mask a macro F1 of just 0.28 on rare classes.

Class-Imbalance Awareness - Uses stratified splitting and class-weighted training to address the 96.5% majority class problem, rather than ignoring it.

Model Comparison Rigor - Systematically compares Decision Tree, Random Forest, and XGBoost, selecting the best performer based on macro F1 rather than raw accuracy.

Explainable AI - SHAP analysis makes the model's reasoning transparent, showing maintenance engineers exactly which sensor readings (Torque, Tool Wear) drive each failure prediction.

Transparent Limitations - Explicitly documents where the model fails (Tool Wear and Random Failures, due to insufficient training samples) rather than hiding weaknesses behind aggregate accuracy.

This makes the solution a **genuinely interpretable and honest maintenance tool**, not just a high-accuracy black box.

Git Hub Link

Github URL – <https://github.com/Vidhya-Majee/predictive-maintenance-ml>

This repository includes:

1. Your notebook (predictive_maintenance.ipynb)
2. The problem statement and final PPT
3. README.md
4. confusion_matrix.png
5. shap_summary.png
6. dataset CSV

.

.

Future Scope

1.SMOTE / Oversampling for Minority Classes –

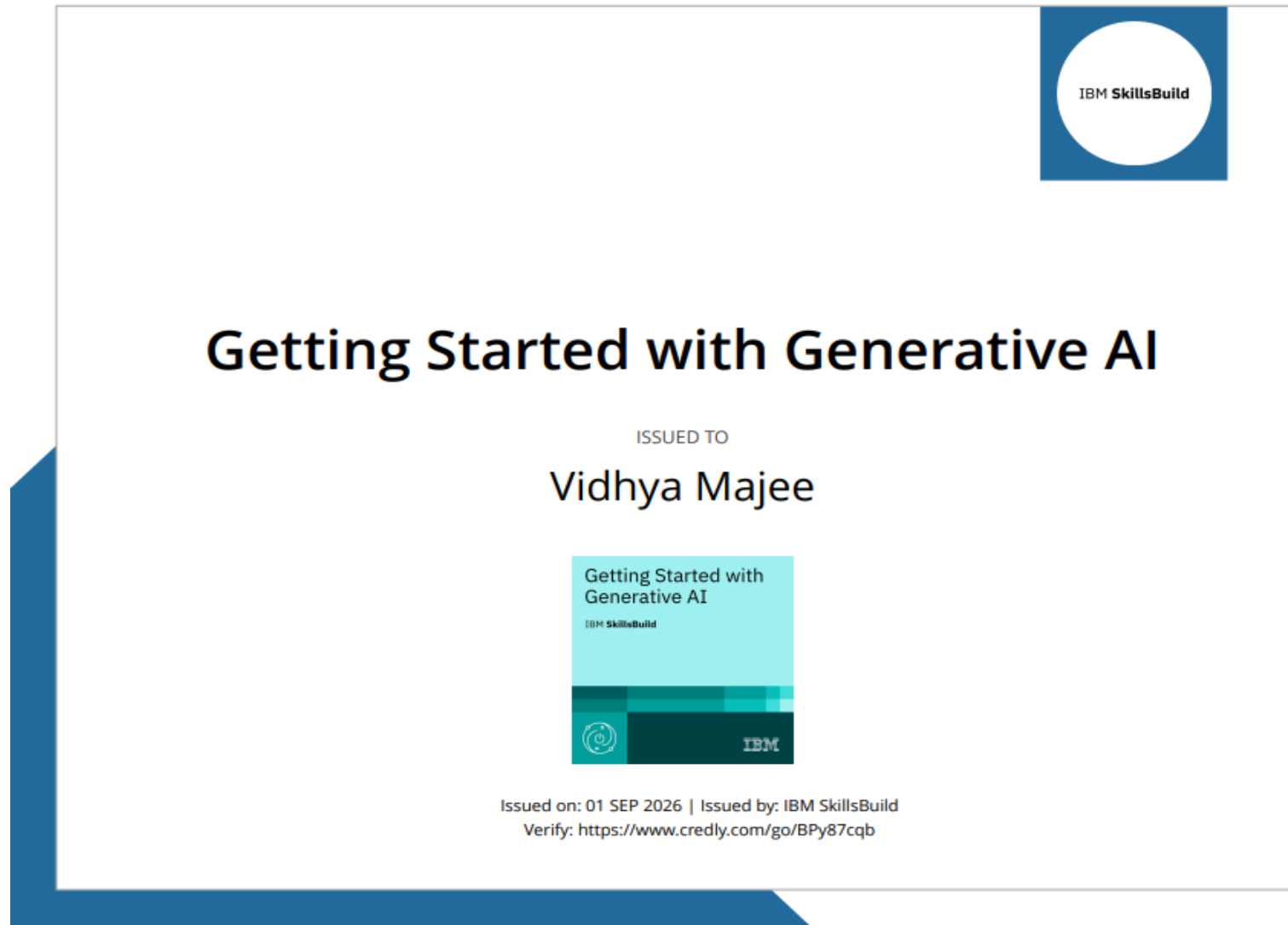
Applying synthetic oversampling techniques (SMOTE/ADASYN) could help the model learn patterns for Tool Wear and Random Failures, which currently have too few samples (45 and 18 respectively) for reliable detection.

2.Real-Time Deployment & Integration –

The trained model could be deployed as a REST API integrated with live sensor streams from industrial equipment, enabling real-time failure alerts rather than batch predictions, and extended with IBM Cloud services once account access is available.

Certificates Earned:

Please add your credly certificate(Getting Started with Artificial Intelligence)



Certificates Earned:

Add your IBM BOB Certificate

IBM **SkillsBuild**

Completion Certificate



This certificate is presented to

Vidhya Majee

for the completion of

Lab: Troubleshoot Your Code Using IBM Bob

(ALM-COURSE_4071307)

According to the Adobe Learning Manager system of record

Completion date: 27 Jul 2026 (GMT)

Learning hours: 30 mins

Thank You!

Thank you for your time and interest.